TECHNICAL REFERENCE

PowerGlove Vision Configuration Reference

Active files, installed copies, fields, secrets, and generated state.

2026 EDITION / IAIN BENNETT / MIT LICENSE

PowerGlove Vision divides configuration between the UNO Q application, the RetroPie receiver, the Arduino sketch, and packaging metadata. This reference identifies the copy that should be edited, the component that reads it, and whether it may contain secrets.

JSON deliberately has no comment syntax. The JSON files remain machine-valid and their field documentation lives here instead of being embedded in them. YAML, TOML, RetroArch, and systemd files support comments, so those files also carry standard source headers.

Configuration map

Repository file	Active or installed copy	Consumer	Secret?
`config/device.example.json`	UNO Q app `data/device.json`	App supervisor and web setup	**Yes**, active copy contains the shared token
`config/games.json`	RetroPie `/etc/powerglove/games.json`	Launch hook and profile selector	No
`config/launcher.example.json`	RetroPie `/etc/powerglove/launcher.json`	RetroPie runcommand hook	No, but it points to the protected token file
`config/profiles.json`	Repository or deployed app copy	Vision worker and gesture engine	No
`app.yaml`	UNO Q App Lab project manifest	App Lab	No
`sketch/sketch.yaml`	UNO Q sketch build manifest	Arduino build service	No
`pyproject.toml`	Repository root	Python installer and packaging tools	No
`retropie/retroarch/PowerGloveVision.cfg`	RetroArch autoconfig directory	RetroArch	No
`retropie/powerglove-receiver.service`	`/etc/systemd/system/`	RetroPie systemd	No
`retropie/powerglove-receiver.timer`	`/etc/systemd/system/`	RetroPie systemd	No
`uno-q/powerglove-system-shutdown.path`	`/etc/systemd/system/`	UNO Q host systemd	No
`uno-q/powerglove-system-shutdown.service`	`/etc/systemd/system/`	UNO Q host systemd	No
`.github/workflows/quality.yml`	GitHub Actions	Hosted CI runners	No

UNO Q device settings

config/device.example.json

This is a public example of the private settings created at `data/device.json`. The App Lab supervisor in `python/main.py` creates the active file on first launch if it is absent. The Setup page reads and updates its non-secret values through the control server.

Field	Meaning
`receiver`	RetroPie hostname or address. `retropieconsole.local` is the portable default.
`token`	Random shared secret used to authenticate controller and profile traffic. The same value must be installed in `/etc/powerglove/token` on RetroPie.
`profile`	Startup profile used before a RetroPie game selects another profile.
`glove_color`	Display hint: `none`, `white`, or `black`. It currently labels diagnostics and does not perform color segmentation.
`camera`	`auto`, a numeric camera index, or an explicit Linux video-device path. `auto` prefers stable USB camera paths.
`port`	Optional controller-packet destination port. It defaults to `55355` when omitted.

Never commit, publish, screenshot, or paste the active `data/device.json`. Deployment and packaging exclude `data/`, so updates preserve device settings and public packages cannot disclose the token.

Automatic per-game profile selection

config/games.json

This registry maps an exact ROM basename to a gesture profile. Install it as `/etc/powerglove/games.json` on RetroPie. Matching is case-insensitive, ignores the directory portion, and currently applies only to NES and Famicom systems.

The `games` object uses this form:

```
JSON
{
  "games": {
    "Super Glove Ball (USA).nes": "super_glove_ball"
  }
}
```

Valid values are `bad_street_brawler`, `super_glove_ball`, and `program_a` through `program_i`. Add the exact filenames present in EmulationStation, including archive extensions such as `.zip` or `.7z` when applicable. An unknown profile makes registry loading fail rather than silently selecting the wrong controls.

Back up a customized installed registry before replacing it during an update. The repository copy provides safe defaults; the installed copy is the cabinet-specific source of truth.

RetroPie launch connection

config/launcher.example.json

Copy this template to /etc/powerglove/launcher.json. The runcommand hook reads it each time a game starts or exits.

Field	Meaning
uno_q	UNO Q hostname or address that receives profile requests. A working .local name is preferred over a changing DHCP address.
port	Authenticated profile-control UDP port; default 55356.
token_file	Protected RetroPie file containing the shared token; normally /etc/powerglove/token.
registry	Installed ROM-to-profile registry; normally /etc/powerglove/games.json.
timeout	Seconds to wait for each acknowledgement attempt. The sender makes up to three bounded attempts.

The launcher file does not contain the secret itself. Keep the token in the separate root-owned file with group-readable permissions for the input service. A missing UNO Q or invalid setting is reported, but the hook intentionally does not prevent the selected game from launching.

Gesture thresholds

config/profiles.json

The vision worker loads this file when a profile starts. Game-specific objects override the built-in GestureConfig defaults. Cartridge-free Programs A-I use program_defaults unless code supplies a specialized mapping.

Field	Meaning
move_on / move_off	Palm displacement, normalized by apparent palm size, that presses and releases a direction.
curl_on / curl_off	Normalized finger curl thresholds from 0.0 straight to 1.0 tightly bent.
roll_on / roll_off	Wrist-roll press and release thresholds measured as a fraction of a quarter turn.
push_on / push_off	Relative apparent-hand-size change used for monocular push depth.
pulse_hz	Pulse frequency used by actions such as Bad Street Brawler's repeated B input.
loss_release_ms	Maximum tracking-loss grace period before every virtual control is released.

For every hysteresis pair, keep the _off value lower than _on. Change one pair at a time, use the dashboard diagnostics, and retain the 120 ms safety release unless camera testing demonstrates a clear need.

UNO Q and Python manifests

`app.yaml`

App Lab reads this manifest to name the application, show its icon, and publish ports `8088` and `8443`. Port `8088` serves the dashboard and diagnostics; `8443` provides the protected HTTPS setup and pairing routes. The deployment script also verifies that App Lab's generated compose file publishes the secure port.

`sketch/sketch.yaml`

The Arduino build service reads this file to select the `arduino:zephyr` platform and exact Router Bridge dependency versions. Change pinned versions only after compiling the sketch and testing matrix status, profile, and pairing displays on the UNO Q.

`pyproject.toml`

Python build tools read this file for package metadata, the `src/` layout, console commands, and optional dependency groups. The base package remains compatible with the older RetroPie Python environment. Camera tracking uses the separate `vision` dependencies, while the receiver uses the `evdev` dependency. The UNO Q App Lab worker deliberately resolves its MediaPipe runtime separately because its Python requirements differ from RetroPie.

RetroArch virtual-controller mapping

`retropie/retroarch/PowerGlove Vision.cfg`

Install this file in RetroArch's autoconfig directory. RetroArch selects it by the uinput device name `PowerGlove Vision`. It maps A, B, Select, Start, D-pad, and four axes to the event codes created by the receiver. The vendor and product IDs are intentionally the receiver's stable virtual IDs, not USB hardware IDs.

If the virtual device name or event layout changes, update the receiver and this file together. Do not use this file to remap the cabinet I-PAC or 8BitDo controllers; their independent profiles remain valid alongside PowerGlove Vision.

systemd runtime configuration

RetroPie receiver service and timer

`retropie/powerglove-receiver.service` runs the privileged receiver that creates the uinput gamepad. It reads `/etc/powerglove/token`, listens on controller port `55355`, restarts after failure, and releases controls when packets stop.

`retropie/powerglove-receiver.timer` starts that service 45 seconds after boot. The delay lets EmulationStation finish its initial controller scan and prevents virtual-device arrival from disturbing the cabinet's merged controller setup. Enable the timer; do not also enable the service directly at boot.

UNO Q shutdown path and service

`uno-q/powerglove-system-shutdown.path` watches for one fixed request file in the application data directory. `uno-q/powerglove-system-shutdown.service` deletes that request and asks systemd to power off Linux without blocking the web response. The installer places both root-owned units on the UNO Q host.

The web application cannot run an arbitrary privileged command. It can only create the fixed request after confirmation, and only when the installed helper has created the private `.shutdown-enabled` marker.

Generated files that are not repository configuration

- `data/device.json` is private persistent UNO Q state and is ignored by Git.
- `data/models/hand_landmarker.task` is a downloaded, checksum-verified Google model and is ignored by Git.
- `.cache/app-compose.yaml` is generated by App Lab on the UNO Q. The deployment script ensures it publishes port `8443`; do not maintain a local copy.
- `data/.shutdown-enabled` is an installation marker created by the shutdown helper, not a user-editable setting.

GitHub Actions workflow

`.github/workflows/quality.yml`

GitHub runs this workflow for pull requests, pushes to `main`, and manual requests. It tests Python 3.7 and 3.12, checks Python and shell syntax, validates JSON, audits source and documentation, rebuilds and inspects all PDF editions, builds the public App Lab package, rejects private or unsafe archive content, and publishes the verified ZIP as a 14-day workflow artifact.

The workflow has read-only repository permissions and uses no project secrets. Keep deployment credentials and live-device operations out of this workflow; deployment remains an explicit maintainer action.

Public package location

Build the importable App Lab package with:

```
SH
scripts/build-app-lab-package.sh
```

The resulting file is:

```
output/app-lab/PowerGlove-Vision-Uno-Q.zip
```

The ZIP is a generated release artifact and is intentionally ignored by Git. It contains the application source, configuration examples, documentation, and the required UNO Q MediaPipe wheel. It excludes private `data/`, the downloaded Google model, tests, PDFs, caches, and Git history. The Google model downloads and verifies itself on first launch.

For a public release, attach the verified ZIP to a tagged GitHub Release. Do not commit changing 33 MB ZIP binaries into the source branch.